

A Symbolic Expression Framework for Structured Computation in Python

Tran Minh Duc Nguyen

1. Background and Domain

1.1 Domain Overview

The focus of this project is symbolic computation and embedded domain-specific languages within Python. In standard Python, expressions are evaluated eagerly, which means computations produce concrete results right after variables are assigned. However, in systems like spreadsheets, query planners, and compilers, computations are initially represented symbolically as abstract syntax trees (ASTs) or dependency graphs before they are executed. This symbolic representation allows for optimization, rewriting, static analysis, and validation before evaluation.

Our project deals with this by creating a symbolic expression framework embedded in Python that builds ASTs rather than performing immediate computations.

1.2 Problem Specification

Python's default evaluation model executes expressions immediately, which means the structure of a computation is lost as soon as it runs. This makes it impossible to analyze, validate, or rewrite expressions before they are executed.

This becomes a real problem in systems where expressions are user-defined or dynamically constructed at runtime, such as formula engines, rule-based systems, or configuration-driven pipelines. Consider a scenario where a user defines a pricing formula like

$$(\text{price} - \text{discount}) / \text{quantity}.$$

The formula is built once but evaluated repeatedly with different data. If quantity happens to be zero, Python crashes mid-execution with no meaningful context. More critically, by the time the crash occurs, partial work may have already been done, such as writing to a database or sending a request.

The problem is further compounded in scientific and engineering contexts where values carry physical units. A velocity reading might arrive as *mph* from one sensor and *km/h* from another and the formula expecting meters silently receives kilometers. Python has no concept of units, it will add meters to kilograms without complaint, producing a result that is numerically computed but physically meaningless. These errors are invisible until they surface as wrong answers, sometimes long after the computation has run.

Our framework addresses this by representing expressions as Abstract Syntax Trees (ASTs) rather than evaluating them immediately. This allows the system to validate the expression against provided data before any execution occurs such as catching errors like division by zero, undefined variables, or invalid inputs to functions like `sqrt` and `log` and unit-conversion incompatibilities and then reporting them with clear, descriptive messages. Only after validation passes does the framework proceed to evaluate the expression.

1.3 Benefits

This framework offers direct benefits to two groups of users. For developers building formula-driven systems such as pricing engines, financial calculators, or rule-based pipelines. The framework will provides a safety layer that validates expressions before execution, catching errors like division by zero, undefined variables, invalid mathematical inputs such as square roots of negative numbers and unit-conversion incompatibilities . This prevents cryptic runtime crashes and ensures that partial side effects, such as database writes or network requests, are not triggered before a formula is confirmed safe.

For scientific and engineering users, the framework adds a layer of physical correctness that plain Python cannot provide. A variable can be declared as meters in an expression with a constant as kilograms and the framework automatically normalizes mixed units within the same dimension before evaluation. So if a formula receives velocity readings in mph, km/h, and m/s from different sources, the framework resolves them to a common base before computing. If the requested target unit is dimensionally incompatible with the result like asking for kilograms when the expression produces meters, the incompatibility is caught at validation time with a clear error message, before any computation runs.

For programming language researchers and students, the framework serves as a transparent and lightweight platform for studying AST transformations and rewrite systems. Because each validation and simplification step is implemented as an independent pass over the AST, the system makes it easy to experiment with new rules without modifying the core framework.

2. Intellectual Merit

There is existing work on symbolic computation and expression evaluation in Python, and several systems operate in adjacent spaces. MEV occupies a deliberately different design space, it focused on safe, validated formula execution with physical unit awareness rather than full algebraic manipulation or hardware-optimized computation.

One of the example library is SymPy, it is a full-featured computer algebra system that is built into Python, it can do symbolic manipulation, solve equations, differentiate, and do more advanced algebraic transformations. However, SymPy is designed as a complete mathematics system, not a lightweight execution framework. It has no concept of a validation-first pipeline, expressions are manipulated symbolically but there is no mechanism to validate an expression against a runtime data context before evaluation, catch unit-conversion incompatibilities, or prevent partial side effects from occurring on bad inputs. MEV targets exactly this gap: not symbolic algebra, but safe and inspectable formula execution.

Another relevant comparison is TensorFlow and similar machine learning frameworks that build computation graphs before execution. These systems make symbolic graphs that can be optimized and run quickly on special hardware. However, these systems are tightly coupled to tensor operations and hardware acceleration. They are not designed for general formula execution, offer no unit awareness, and provide no human-readable validation diagnostics. MEV borrows the idea of separating representation from execution but applies it to a completely different problem domain.

Microsoft Excel and other spreadsheet programs also use dependency graphs to show formulas and automatically recalculate values when the inputs change. They let users track dependencies, but they don't offer a programmable, extensible rewriting framework. We took this idea and made it into a programmable symbolic infrastructure built into Python.

Beyond these comparisons, MEV makes three concrete contributions. First, it provides a structured validation pipeline that walks the full AST before evaluation and collects all errors in a single pass like undefined variables, division by zero, domain violations for functions like `sqrt` and `log`, and unit-conversion incompatibilities, then reporting them with structured, human-readable messages before any side effects occur. Second, it integrates physical unit awareness directly into the expression layer, so values carry unit intent from declaration through evaluation, mixed units are normalized automatically, and target unit compatibility is validated before the expression runs. Third, it cleanly separates representation, validation, and evaluation into distinct phases, which makes the system lightweight and extensible, new node types, new validation rules, and new math functions can be added independently without touching the core framework

3. Action Plan

3.1 Expression Syntax

The framework is embedded directly in Python and exposes its interface through a Python API using operator overloading. Users construct symbolic expressions by declaring

variables and constants, then combining them with supported operations. The result is a natural, readable expression syntax that builds an AST rather than computing a value.

Variables are declared with `var()` and constants with `val()`. Both accept an optional unit argument that declares the physical dimension of the value. When a unit is declared, the framework enforces it at validation time, if the runtime context provides a value in an incompatible unit, the incompatibility is caught before evaluation.

Unit-aware evaluation is handled through Pint. When an expression involves values with different but compatible units such as meters and kilometers, or mph and m/s, the framework normalizes them to a common base unit before performing arithmetic. The caller can optionally specify a target unit at evaluation time, and the framework will validate that the result dimension is compatible before converting.

3.2 Data Structures and Core Algorithms

The core data structure will be a tree-based Abstract Syntax Tree. All symbolic expressions will inherit from a base `Expr` class. Concrete subclasses will represent different node types such as variables, constants, and binary operations. Overall, we will implement this with Composite design pattern.

4. Limitations and Future Work

This framework is really limited in scope. It does not model full Python semantics such as control flow, side effects, or higher-order functions. The rewrite system relies on manually defined transformation rules, which means simplification is not guaranteed to be complete or optimal. Additionally, static validation features will be limited to basic syntactic checks rather than formal correctness proofs, and the system is not designed for large-scale performance optimization.

In future work we could extend the framework with a lightweight type system to provide stronger static guarantees, more advanced rewriting strategies such as normalization or equivalence checking, and support for additional language constructs. The system could also be expanded with compilation backends or visualization tools for dependency graphs, improving both performance and usability.